Ch2 Lecture 3

Difference Equations

Difference Equations

Example: Digital Filters

Digital Filters

Another use for difference equations:

$$y_k = a_0 x_k + a_1 x_{k-1} + \cdots + a_m x_{k-m}, \quad k = m, m+1, m+2, \ldots,$$

. . .

x is a continuous variable – perhaps sound in time domain, or parts of an image in space.

y is some filtered version of x.

Example: Cleaning up a noisy signal

True signal: $f(t) = \cos(\pi t), -1 \leq t \leq 1$

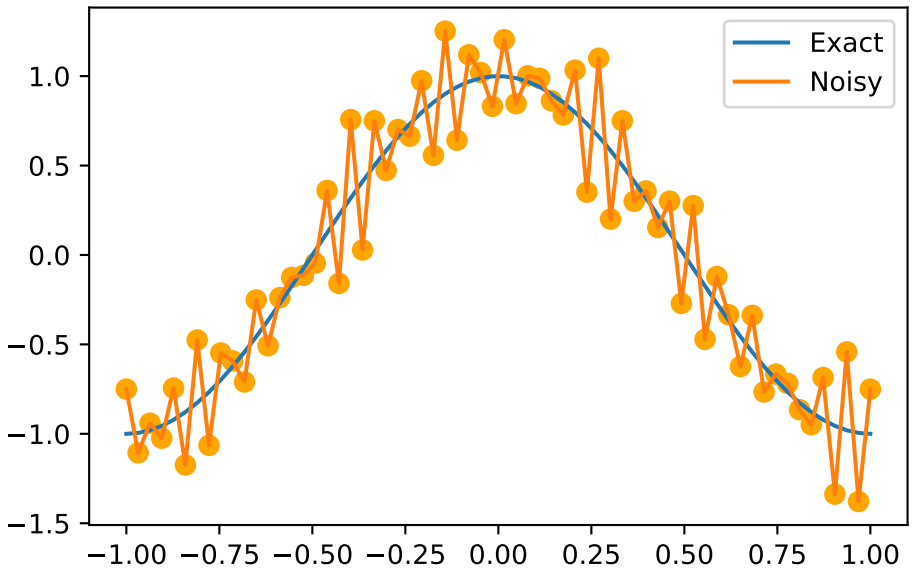
Signal plus noise: $g(t) = \cos(\pi t) + \frac{1}{5}\sin(24\pi t) + \frac{1}{4}\cos(30\pi t)$

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(start=-1,stop=1,num=64)
def f(x):
    return np.cos(np.pi*x)
def g(x):
```

```
    return np.cos(np.pi*x) + 1/5*np.sin(24*np.pi*x) + 1/4*np.cos(30*np.pi*x)

plt.plot(x, f(x), label='Exact')
plt.plot(x, g(x), label='Noisy')
plt.scatter(x, g(x), color='orange', s=50)  # Add dots for 'Noisy'
plt.legend()
```



Filtered data

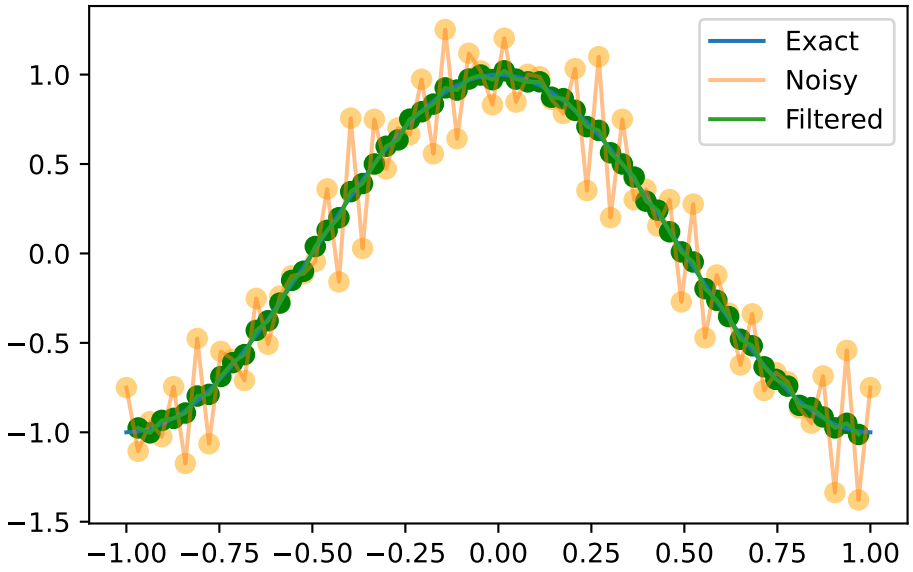
Idea: we can sample nearby points and take a weighted average of them.

$$y_k = \frac{1}{4}x_{k+1} + \frac{1}{2}x_k + \frac{1}{4}x_{k-1}, \quad k = 1, 2, \dots, 63$$

. . .

```
y = np.zeros_like(x)
for k in range(1, 63):
    y[k] = 1/4*g(x[k+1]) + 1/2*g(x[k]) + 1/4*g(x[k-1])
plt.plot(x, f(x), label='Exact')
plt.plot(x, g(x), label='Noisy', alpha=0.5)
plt.scatter(x, g(x), color='orange', s=50, alpha=0.5)  # Add dots for 'Noisy'
plt.plot(x[1:-1], y[1:-1], label='Filtered')
```

```
plt.scatter(x[1:-1], y[1:-1], color='green', s=50)  # Add dots for 'Filtered'
plt.legend()
```



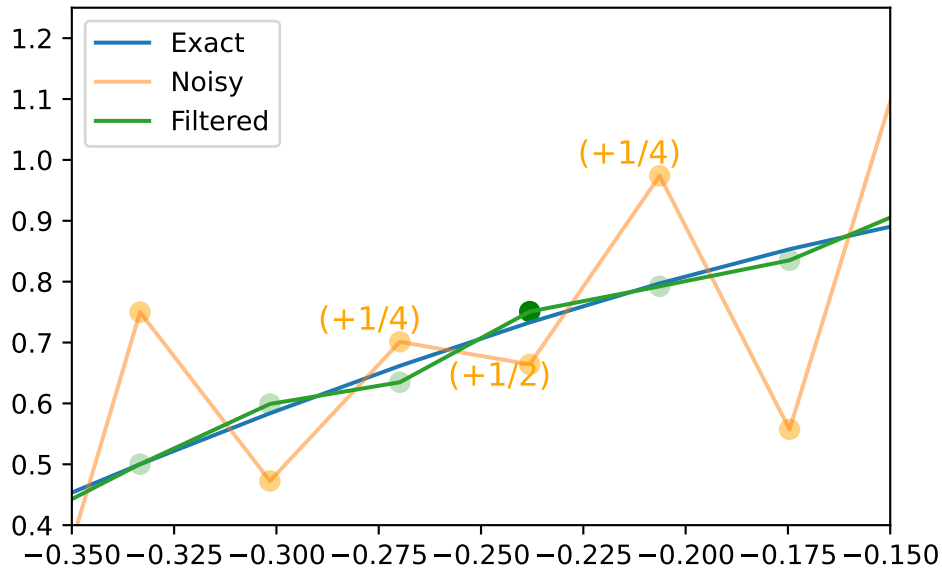
. . .

This captured the low-frequency part of the signal, and filtered out the high-frequency noise. That's just what we wanted! This is called a *low-pass filter*.

We can zoom in on a few points to see the effect of the filter.

```
plt.plot(x, f(x), label='Exact')
plt.plot(x, g(x), label='Noisy', alpha=0.5)
plt.scatter(x, g(x), color='orange', s=50, alpha=0.5)  # Add dots for 'Noisy'
plt.plot(x, y, label='Filtered')
for i in range(len(x)):
  if i == 24:
    plt.scatter(x[i], y[i], color='green', s=50, alpha=1)  # Add dot for y[24] with 100% opa
  else:
    plt.scatter(x[i], y[i], color='green', s=50, alpha=0.25)  # Add other dots for 'Filtered
plt.text(x[25]-.02, g(x[25])+.02, '(+1/4)', color='orange', fontsize=12)  # Add label to the
plt.text(x[24]-.02, g(x[24])-.035, '(+1/2)', color='orange', fontsize=12)  # Add label to th
plt.text(x[23]-.02, g(x[23])+.02, '(+1/4)', color='orange', fontsize=12)  # Add label to the
```

```
plt.xlim(-0.35, -.15)
# make the ylim automatically adjust to the data in the zoomed in region
plt.ylim(bottom=0.4, top=1.25)
plt.legend()
```



Now you try

See if you can make a figure which will just find the noisyness...

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(start=-1,stop=1,num=64)
def f(x):
    return np.cos(np.pi*x)
def g(x):
    return np.cos(np.pi*x) + 1/5*np.sin(24*np.pi*x) + 1/4*np.cos(30*np.pi*x)
y = np.zeros_like(x)
for k in range(1, 63):
    y[k] = 1/4*g(x[k+1]) + 1/2*g(x[k]) + 1/4*g(x[k-1])
fig = plt.figure()
plt.clf()
plt.plot(x, f(x), label='Exact')
```

```
plt.plot(x, g(x), label='Noisy', alpha=0.5)
plt.scatter(x, g(x), color='orange', s=50, alpha=0.5)  # Add dots for 'Noisy'
plt.plot(x[1:-1], y[1:-1], label='Filtered')
plt.scatter(x[1:-1], y[1:-1], color='green', s=50)  # Add dots for 'Filtered'
plt.legend()
plt.show()
```

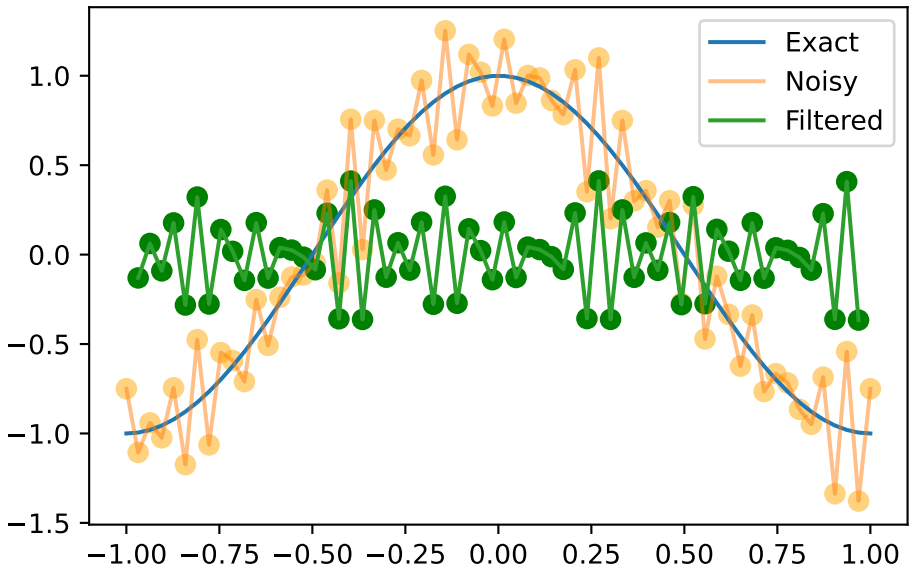
A different filter

What if we *subtract* the values at x_{k+1} and x_{k-1} instead of adding them?

$$y_k = -\frac{1}{4}x_{k+1} + \frac{1}{2}x_k - \frac{1}{4}x_{k-1}, \quad k = 1, 2, \ldots, 63$$

. . .

```
y = np.zeros_like(x)
for k in range(1, 63):
    y[k] = -1/4*g(x[k+1]) + 1/2*g(x[k]) - 1/4*g(x[k-1])
plt.plot(x, f(x), label='Exact')
plt.plot(x, g(x), label='Noisy', alpha=0.5)
plt.scatter(x, g(x), color='orange', s=50, alpha=0.5)  # Add dots for 'Noisy'
plt.plot(x[1:-1], y[1:-1], label='Filtered')
plt.scatter(x[1:-1], y[1:-1], color='green', s=50)  # Add dots for 'Filtered'
plt.legend()
```



. . .

This is a *high-pass filter*. It captures the high-frequency noise, but filters out the low-frequency signal.

Inverse Matrices

Definition of inverse matrix

Let A be a square matrix.

Inverse for A is a square matrix B of the same size as A

- such that $AB = I = BA$.
-
- If such a B exists, then the matrix A is said to be **invertible**.
- Also called **"singular"** (non-invertible), or **"nonsingular"** (invertible)

Conditions for invertibility

Conditions for Invertibility The following are equivalent conditions on the square $n \times n$ matrix A :

1. The matrix A is invertible.
2. There is a square matrix B such that $BA = I$.
3. The linear system $A\mathbf{x} = \mathbf{b}$ has a unique solution for every right-hand-side vector $\mathbf{b}$.
4. The linear system $A\mathbf{x} = \mathbf{b}$ has a unique solution for some right-hand-side vector $\mathbf{b}$.
5. The linear system $A\mathbf{x} = 0$ has only the trivial solution.
6. $\operatorname{rank} A = n$.
7. The reduced row echelon form of A is I_n.
8. The matrix A is a product of elementary matrices.
9. There is a square matrix B such that $AB = I$.

Elementary matrices

- Each of the operations in row reduction can be represented by a matrix E.

. . .

Remember: - E_{ij} : The elementary operation of switching the ith and jth rows of the matrix. - $E_i(c)$: The elementary operation of multiplying the ith row by the nonzero constant c. - $E_{ij}(d)$: The elementary operation of adding d times the jth row to the ith row.

. . .

Find an **elementary matrix** of size n is by performing the corresponding elementary row operation on the identity matrix I_n.

. . .

Example: Find the elementary matrix for $E_{13}(-4)$

. . .

Add -4 times the 3rd row of I_3 to its first row...

. . .

$$E_{13}(-4) = \begin{bmatrix} 1 & 0 & -4 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Recall an example from the first week:

Solve the simple system

$$2x - y = 14x + 4y = 20. \tag{1.5}$$

. . .

$$\begin{bmatrix} 2 & -1 & 1 \\ 4 & 4 & 20 \end{bmatrix} \overrightarrow{E_{12}} \begin{bmatrix} 4 & 4 & 20 \\ 2 & -1 & 1 \end{bmatrix} \overrightarrow{E_1(1/4)} \begin{bmatrix} 1 & 1 & 5 \\ 2 & -1 & 1 \end{bmatrix}$$
$$\overrightarrow{E_{21}(-2)} \begin{bmatrix} 1 & 1 & 5 \\ 0 & -3 & -9 \end{bmatrix} \overrightarrow{E_2(-1/3)} \begin{bmatrix} 1 & 1 & 5 \\ 0 & 1 & 3 \end{bmatrix} \overrightarrow{E_{12}(-1)} \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \end{bmatrix}.$$

Rewrite this using matrix multiplication:

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \end{bmatrix} = E_{12}(-1)E_2(-1/3)E_{21}(-2)E_1(1/4)E_{12} \begin{bmatrix} 2 & -1 & 1 \\ 4 & 4 & 20 \end{bmatrix}.$$

Remove the last columns in each matrix above. (They were the "augmented" part of the original problem.) All the operations still work:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = E_{12}(-1)E_2(-1/3)E_{21}(-2)E_1(1/4)E_{12} \begin{bmatrix} 2 & -1 \\ 4 & 4 \end{bmatrix}.$$

. . .

Aha! We now have a formula for the inverse of the original matrix:

$$A^{-1} = E_{12}(-1)E_2(-1/3)E_{21}(-2)E_1(1/4)E_{12}$$

Superaugmented matrix

- Form the **superaugmented matrix** $[A \mid I]$.
- If we perform the elementary operation E on the superaugmented matrix, we get the matrix E in the augmented part:

. . .

$$E[A \mid I] = [EA \mid EI] = [EA \mid E]$$

. . .

- This can help us keep track of our operations as we do row reduction
- The augmented part is just the product of the elementary matrices that we have used so far.
- Now continue applying elementary row operations until the part of the matrix originally occupied by A is reduced to the reduced row echelon form of A.

. . .

$$[A \mid I]\overrightarrow{E_1, E_2, \ldots, E_k}[I \mid B]$$

- $B = E_k E_{k-1} \cdots E_1$ is the product of the various elementary matrices we used.

Inverse Algorithm

Given an $n \times n$ matrix A, to compute A^{-1} :

1. Form the superaugmented matrix $\widetilde{A} = [A \mid I_n]$.
2. Reduce the first n columns of $\widetilde{A}$ to reduced row echelon form by performing elementary operations on the matrix $\widetilde{A}$ resulting in the matrix $[R \mid B]$.
3. If $R = I_n$ then set $A^{-1} = B$; otherwise, A is singular and A^{-1} does not exist.

2x2 matrices

Suppose we have the 2x2 matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

. . .

Do row reduction on the superaugmented matrix:

$$\left[\begin{array}{cc|cc} a & b & 1 & 0 \\ c & d & 0 & 1 \end{array}\right]$$

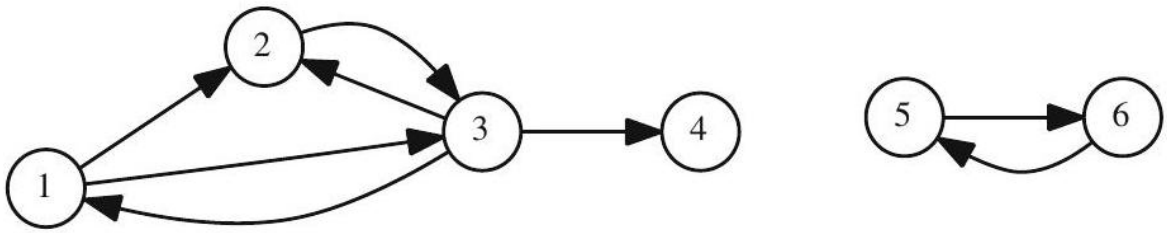
. . .

```
import sympy as sym
a, b, c, d = sym.symbols('a b c d')
A = sym.Matrix([[a, b], [c, d]])
augmented = A.row_join(sym.eye(2))
reduced = augmented.rref()
reduced
```

$$A^{-1} = \frac{1}{D}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

PageRank revisited

Back to PageRank



See the animated power iteration (x_{n+1}=Qx_n) on this graph: PageRank animation page.

Do page ranking as in the first week:

- for page j let n_j be its total number of outgoing links on that page.
- Then the score for vertex i is the sum of the scores of all vertices j that link to i, divided by the total number of outgoing links on page j. . . .

$$x_i = \sum_{x_j \in L_i} \frac{x_j}{n_j}$$

$$\begin{aligned}
x_1 &= \frac{x_3}{3} \\
x_2 &= \frac{x_1}{2} + \frac{x_3}{3} \\
x_3 &= \frac{x_1}{2} + \frac{x_2}{1} \\
x_4 &= \frac{x_3}{3} \\
x_5 &= \frac{x_6}{1} \\
x_6 &= \frac{x_5}{1}
\end{aligned}$$

PageRank as a matrix equation

Define the matrix Q and vector $\mathbf{x}$ by

$$Q = \begin{bmatrix} 0 & 0 & \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{2} & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{3} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}, \mathbf{x} = (x_1, x_2, x_3, x_4, x_5)$$

. . .

Then the equation $x = Qx$ is equivalent to the system of equations above.

. . .

$\mathbf{x}$ is a **stationary vector** for the transition matrix Q.

Connection between adjacency matrix and transition matrix

- A: adjacency matrix of a graph or digraph
- D: be a diagonal matrix whose i th entry is either:
  - the inverse of the sum of all entries in the i th row of A, or
  - zero if if this sum is zero.
- Then $Q = A^T D$ is the transition matrix for the page ranking of this graph.

Check.

Remember, for our simple network example, we had the adjacency matrix:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

```
import sympy as sym
A = sym.Matrix([[0, 1, 1, 0, 0, 0], [0, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 0, 0, 0, 0
D = sym.diag(sym.Rational(1,3), sym.Rational(1,2), sym.Rational(1,3), 0, 1, 1)
Q = A.T*D
Q
```

PageRank as a Markov chain

Think of web surfing as a **random process**

- each page is a state
-
- probabilities of moving from one state to another given by a stochastic transition matrix P.
-
- equal probability of moving to any of the outgoing links of a page

. . .

$$P \stackrel{?}{=} \begin{bmatrix} 0 & 0 & \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{2} & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{3} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Pause: Is P a stochastic matrix?

Correction vector

We can introduce a *correction vector*, equivalent to adding links from the dangling node to every other node, or to all connecting nodes (via some path).

. . .

$$P = \begin{bmatrix} 0 & 0 & \frac{1}{3} & \frac{1}{3} & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{3} & \frac{1}{3} & 0 & 0 \\ \frac{1}{2} & 1 & 0 & \frac{1}{3} & 0 & 0 \\ 0 & 0 & \frac{1}{3} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

. . .

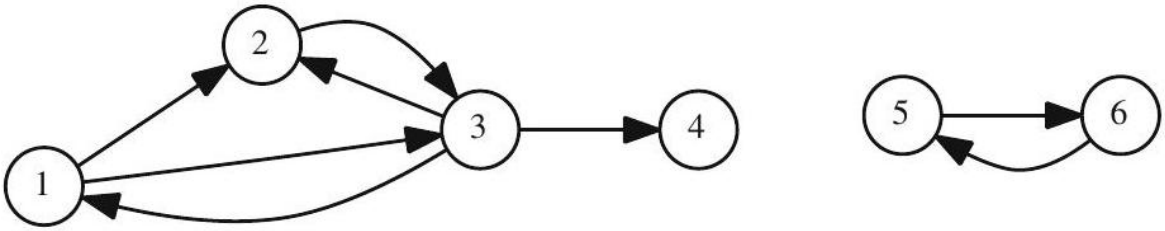
This is now a stochastic matrix **"surfing matrix"**

. . .

Find the new transition matrix Q...

```
A = sym.Matrix([[0, 1, 1, 0, 0, 0], [0, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 0, 0, 0, 0
D = sym.diag(sym.Rational(1,3), sym.Rational(1,2), sym.Rational(1,3), 0, 1, 1)
Q = A.T*D
Q
```

Teleportation



There may not be a single stationary vector for the surfing matrix P.

. . .

Solution: Jump around. Every so often, instead of following a link, jump to a *random page*.

. . .

How can we modify the surfing matrix to include this?

Introduce a **teleportation matrix** E, which is a transition matrix that allows the surfer to jump to any page with equal probability.

Pause: What must E be?

. . .

- Let $\mathbf{v}$ be the teleportation vector with entries 1/n -Let $\mathbf{e} = (1, 1, \ldots, 1)$ be the vector of all ones.
- Then $\mathbf{v}\mathbf{e}^T$ is a stochastic matrix whose columns are all equal to $\mathbf{v}$. This is E.

PageRank Matrix

Let:

- P be a stochastic matrix,
- $\mathbf{v}$ a distribution vector of compatible size
- α a teleportation parameter with $0 < \alpha < 1$.
- Then $\alpha P + (1 - \alpha)\mathbf{v}\mathbf{e}^T$ and
- . . . our goal is to find stationary vectors

$$(\alpha P + (1 - \alpha)\mathbf{v}\mathbf{e}^T)\,\mathbf{x} = \mathbf{x} \tag{2.4}$$

. . .

Rearranging,

$$(I - \alpha P)\mathbf{x} = (1 - \alpha)\mathbf{v} \tag{2.5}$$

Now solve pagerank for our example

Use `M.solve(b)`...

```
A = sym.Matrix([[0, 1, 1, 0, 0, 0], [0, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 0, 0, 0, 0
D = sym.diag(sym.Rational(1,3), sym.Rational(1,2), sym.Rational(1,3), 0, 1, 1)
Q = A.T*D
Q
```